

EDGE AI FOR ATTENTIONAL GUIDANCE: A LOCALIZED ANDROID SYSTEM FOR SENSORY-AWARE ASD SUPPORT

**by
Nika Golestani**

Engineering Project Report

**Yeditepe University
Faculty of Engineering
Department of Computer Engineering
2026**

ACKNOWLEDGEMENTS

I would like to thank my supervisor, Prof. Dr. Emin Erkan Korkmaz, for his guidance and patience throughout this project, and Dr. Öğr. Üyesi Zuhar Rende Berman for generously sharing her perspective as a special education specialist.

This work is dedicated to my brother and to every child whose mind works differently in a world that wasn't designed with them in mind, and to every family fighting to make that world a little better for them.

ABSTRACT

EDGE AI FOR ATTENTIONAL GUIDANCE: A LOCALIZED ANDROID SYSTEM FOR SENSORY-AWARE ASD SUPPORT

This thesis presents a real-time attentional guidance system designed to estimate and enhance task-relevant targets in video content for children with Autism Spectrum Disorder (ASD). The primary objective is to assist users in extracting socially and linguistically relevant information using a lightweight Edge AI pipeline that represents gaze as a spatiotemporal saliency distribution. By subtly enhancing these targets, the system reduces the cognitive load required to identify critical cues in complex media. As a secondary priority, the system incorporates sensory-aware filtering to mitigate perceptual noise, such as luminance spikes and acoustic peaks, which often trigger overstimulation. By utilizing Edge AI rather than cloud-based or standard non-cloud solutions, the application ensures real-time performance and universal accessibility on standard Android hardware, providing high-quality attentional support entirely offline.

ÖZET

DİKKAT YÖNLENDİRMESİ İÇİN UÇ YAPAY ZEKA: DUYU-FARKINDALIKLI OSB DESTEĞİ İÇİN YEREL BİR ANDROİD SİSTEMİ

Bu tez, Otizm Spektrum Bozukluğu (OSB) olan çocuklar için video içeriklerindeki görevle ilgili hedefleri tahmin etmek ve görsel olarak desteklemek amacıyla tasarlanmış gerçek zamanlı bir dikkat yönlendirme sistemi sunmaktadır. Temel amaç, bakışı uzamsal-zamansal (spatiotemporal) bir belirginlik (saliency) dağılımı olarak modelleyen hafif bir Uç Yapay Zeka (Edge AI) işlem hattı kullanarak, kullanıcıların sosyal ve dilbilimsel açıdan önemli bilgileri daha rahat seçmelerine yardımcı olmaktır. Sistem, bu hedefleri görsel olarak ince bir şekilde belirginleştirerek, karmaşık medyalarındaki kritik ipuçlarını algılamak için gereken bilişsel yükü azaltmaktadır. İkincil bir öncelik olarak bu sistem, sıklıkla aşırı duyu uyartılamayı (overstimulation) tetikleyen ani parlaklık artışları ve akustik tepe noktaları gibi algısal gürültüleri hafifletmek üzere duyu-farkındalıklı (sensory-aware) bir filtreleme mekanizması içermektedir. Uygulama, bulut tabanlı veya standart yerel çözümler yerine doğrudan cihaz üzerinde çalışan Uç Yapay Zeka teknolojisini kullanarak, hem eşzamanlı (gerçek zamanlı) veri işlemeyi garanti altına almakta hem de video verilerini dışarıya aktarmayarak kullanıcı gizliliğini en üst düzeyde korumaktadır.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	iii
ACKNOWLEDGEMENTS	iii
ABSTRACT	iv
ÖZET	v
LIST OF FIGURES	viii
LIST OF TABLES	x
List of Algorithms	xi
LIST OF ABBREVIATIONS	xii
LIST OF ABBREVIATIONS	xii
1. Introduction	1
1.1. Problem Definition	1
1.2. Objectives	1
1.3. Proposed Solution	2
1.4. Contributions	2
1.4.1. Functional Requirements	2
1.4.2. Non-Functional Requirements	3
1.5. Thesis Organization	3
2. BACKGROUND AND RELATED WORK	4
2.1. ASD and Sensory Challenges	4
2.2. Saliency Prediction	5
2.3. Existing Assistive Solutions	6
3. ANALYSIS AND DESIGN	8
3.1. System Overview	8
3.2. Dual-User Architecture	8
3.3. Video Lifecycle	9
3.4. Structural Design	10
3.5. Attentional Guidance Pipeline	12
4. MODEL ARCHITECTURE AND TRAINING	14
4.1. Model Evolution and Architecture Design	14
4.2. Structural Components	15
4.3. The Loss Function	17
4.4. Training	18
4.5. Compression and Quantization	19
4.6. Model Evaluation	19
4.6.1. Evaluation Metrics	20

4.6.2.	Comparison with Competing Methods	24
4.6.3.	Performance Analysis Across Video Categories	25
5.	System Implementation and User Interface Design	26
5.1.	Role Separation and Access Control	26
5.2.	General Settings	27
5.3.	Sensory Calculation and Performance Testing	27
5.4.	Overlay Calculation Engine	30
5.4.1.	Runtime Bottleneck and Inference Rate Control	31
5.4.2.	Emulator Profiling	31
5.5.	Overlay Filters	33
5.6.	Video Ingestion, Analysis Interface, and Front-end	33
6.	Testing and Results	35
6.1.	Machine Learning Model Evaluation	35
6.2.	Sensory-Aware Filtering Results	36
6.3.	System Performance and Integration	36
6.3.1.	Requirements Compliance Summary	36
7.	Conclusion and Future Work	38
	Bibliography	39

LIST OF FIGURES

Figure 3.1.	Video life cycle flow.	10
Figure 3.2.	High-level structural design of the proposed system using a class diagram.	12
Figure 3.3.	Sequence diagram from model input to overlay output.	13
Figure 4.1.	Simplified Model Architecture.	16
Figure 4.2.	Training and validation loss during training.	18
Figure 4.3.	Model size reduction after float16 quantization.	19
Figure 4.4.	Distribution of CC values across the AVIMOS test set.	20
Figure 4.5.	Distribution of SIM values across the AVIMOS test set.	21
Figure 4.6.	Relationship between CC and SIM values.	22
Figure 4.7.	Distribution of NSS values across the AVIMOS test set.	23
Figure 4.8.	Distribution of Recall values across the AVIMOS test set.	23
Figure 4.9.	Distribution of Area Percentage values across the AVIMOS test set.	24
Figure 5.1.	Password-protected access screens for caregiver settings.	26
Figure 5.2.	General settings interface for filter configuration, FPS control, and recommendation settings.	27
Figure 5.3.	Distribution of automated sensory scores across video categories (Ghibli, ASMR Slimes, and Trailers).	30
Figure 5.4.	Comparison between (left) raw saliency heatmap output and (right) final stabilized overlay after the processing pipeline.	31

Figure 5.5.	Android emulator performance monitor. Left: active video playback with 5 FPS saliency inference running — total CPU 303.54% (reported as aggregate across four virtual cores), resident memory 4883.78 MB. Right: video paused, inference loop suspended — total CPU 46.56%, resident memory 4793.83 MB.	32
Figure 5.6.	Visualization of implemented overlay filters.	33
Figure 5.7.	Left: Videos tab showing the uploaded video list, safety scores, and exclusion controls. Centre: Analysis dashboard showing feature distributions and interaction metrics. Right: Front-end gallery providing a distraction-free selection interface for the primary user.	34

LIST OF TABLES

Table 4.1.	AIM 2024 leaderboard comparison using standard saliency metrics. . .	24
Table 4.2.	Comparison of lightweight saliency prediction models and parameter efficiency.	25
Table 4.3.	Performance of the proposed model across different video categories in the AVIMOS test set.	25
Table 6.1.	Requirements compliance summary.	37

List of Algorithms

1	Correlation Coefficient (CC) Calculation	20
2	Similarity (SIM) Calculation	21
3	Normalized Scanpath Saliency (NSS) Calculation	22
4	Recall and Area Percentage Calculation	23
5	Flicker	28
6	Hard-Cut Frequency	28
7	Motion, Color Variance, Visual Complexity	28

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
APK	Android Application Package
API	Application Programming Interface
ASD	Autism Spectrum Disorder
ASMR	Autonomous Sensory Meridian Response
BCE	Binary Cross-Entropy
CC	Correlation Coefficient
CNN	Convolutional Neural Network
CPU	Central Processing Unit
ER	Entity-Relationship
FPS	Frames Per Second
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
KLD	Kullback–Leibler Divergence
KSS	KLD–SIM–CC (combined saliency metric set)
ML	Machine Learning
NSS	Normalized Scanpath Saliency
OS	Operating System
RGB	Red, Green, Blue (Color Model)
SHA	Secure Hash Algorithm
SIM	Similarity
SPD	Sensory Processing Disorder
TFLite	TensorFlow Lite
UI	User Interface
UML	Unified Modeling Language

1. Introduction

This chapter presents the motivation behind the research, the problem it addresses, and the proposed approach. It also outlines the contributions of the work and the structure of the thesis.

1.1. Problem Definition

According to the CDC, approximately 1 in every 36 children is diagnosed with ASD [1]. The condition is primarily associated with difficulties in social communication, and children with ASD often focus on non-social details in their environment rather than on the cues that carry social meaning. Specialist interventions such as Applied Behavior Analysis can be effective, but the cost and the need for one-on-one delivery put them out of reach for a large portion of affected families.

Children with ASD are also frequently drawn to screens. Fridberg et al. [2] showed that standard video content can slow language development in this group because the child does not know where to look. A character may be naming an object while the child's gaze is on something irrelevant in the background. Ekici [3] describes the underlying mechanism: autistic children tend to fixate on borders and non-social elements because complex visual scenes are processed without the filtering that neurotypical viewers apply automatically.

The Gap: There is currently no widely available automated tool that can take any video and filter it to make social and linguistic cues easier to notice while simultaneously reducing overwhelming visual noise.

1.2. Objectives

Wang et al. [4], [5] showed that adding gaze-contingent cues to video improved social attention outcomes in children with ASD. The objective of this thesis is to answer a practical follow-up question: can the same effect be delivered on a standard Android device, for any video, at no recurring cost to the family?

1.3. Proposed Solution

The most direct solution would be to detect faces, track gaze, identify motion, and combine the outputs. The problem is that this quickly becomes a long list of models, each with its own failure modes, and the list never ends — what about animals, or objects a child is fixating on for other reasons. Saliency prediction sidesteps this by learning directly from where humans actually look. One model, one input, one output. This is what makes the approach deployable on a mobile device.

The proposed system is an Android application that runs a saliency model entirely on-device, overlays the predicted regions on the video during playback, and gives the caregiver a per-video breakdown of sensory load before the video reaches the child.

1.4. Contributions

This research intends to bridge the research-to-deployment gap through the demonstration of how computer vision can solve this problem effectively and efficiently, while at the same time ensuring the user's privacy.

Firstly, a lightweight saliency model is used, which is trained to run locally. Secondly, the system performs analysis of video content and also analyzes the viewing pattern of the child to enable parents to understand their preferences and necessities, especially when the child is not capable of communicating such needs clearly.

In this solution, computer vision is used to estimate visual attention and produce saliency maps representing areas of visual attention. This is implemented as a standalone Android application that performs all operations locally.

1.4.1. Functional Requirements

FR-01 (Video Import): The system shall allow the secondary user to upload or select local video files for processing and playback.

FR-02 (Video Deletion): The system shall allow the secondary user to delete imported or processed videos to manage storage efficiently.

FR-03 (Sensory Attribute Display): The system shall analyze and display sensory attributes such as luminance spikes and visual noise before playback.

FR-04 (Filter Configuration): The system shall allow the secondary user to configure therapeutic filters such as saliency highlighting or noise reduction.

FR-05 (Analysis Parameterization): The system shall allow the secondary user to define the FPS rate for AI analysis in order to balance precision and battery consumption.

1.4.2. Non-Functional Requirements

NFR-01 (Real-Time Performance): The AI model shall process video frames in real-time without noticeable latency.

NFR-02 (Smooth Playback): The system shall maintain smooth video playback while applying attentional guidance and sensory filtering.

NFR-03 (Privacy and Offline Operation): The system shall perform all processing on-device without transmitting video data to external servers, ensuring both user privacy and functionality without network connectivity.

1.5. Thesis Organization

The rest of this thesis proceeds in the following manner. Chapter 2 gives an overview of literature review pertaining to ASD, saliency prediction, edge computing, and existing assistive technology systems. Chapter 3 gives the system analysis and design, along with the UML diagram. Chapter 4 explains the architecture of the AI model, as well as its training and evaluation process. Chapter 5 shows the implementation of the proposed solution on Android. Chapter 6 discusses the results of the tests performed for both AI model and Android system.

2. BACKGROUND AND RELATED WORK

This chapter presents the background required to understand the proposed attentional guidance system. First, ASD and sensory processing challenges are discussed to explain why such a system may be useful. Then, the main technical components of the solution are introduced.

2.1. ASD and Sensory Challenges

As mentioned by Ekici [3], individuals with Autism Spectrum Disorder (ASD) may experience difficulties interpreting and filtering sensory information. It is also important to note that autism is a spectrum disorder, meaning that each individual may respond differently to sensory and social stimuli.

Individuals with ASD may find media and environmental stimuli overwhelming. Ekici [3] explains this with a driving example: under normal conditions, people usually ignore unnecessary signs and focus only on important information. However, during heavy snow or storm conditions, people tend to pay attention to every possible sign in the environment. Similarly, autistic individuals may attempt to process many visual and social cues at the same time, which can become mentally exhausting. Because of this, they may avoid or miss important cues such as eye contact, facial expressions, or socially relevant objects.

Research on sensory processing in ASD further confirms that such perceptual differences have measurable neurophysiological correlates [6]. The phenomenon of sensory over-responsivity — where ordinary stimuli produce unusually strong physiological reactions — has been documented across visual, auditory, and tactile modalities. In the visual domain specifically, hypersensitivity to luminance changes and high-contrast edges is particularly relevant to screen-based media, where rapid cuts, flicker, and bright highlights occur frequently. These features are not inherently problematic for neurotypical viewers, but for a child with ASD they may trigger a stress response that interrupts the viewing experience entirely.

Alongside sensory over-responsivity, atypical visual attention is a well-documented characteristic of ASD. Eye-tracking studies have consistently shown that individuals with ASD spend less time looking at faces and socially relevant regions of a scene, and more time on peripheral or non-social details [3]. This pattern does not reflect a simple preference; it appears

to reflect a difference in the automatic salience computation that guides where the eyes move next. For a child watching an educational video, this means that the information the video is designed to convey may pass unnoticed while the child’s gaze is fixed on an irrelevant background element.

Since autism is a spectrum disorder, sensory sensitivities and attentional patterns may differ significantly between children. Because of this, defining a single fixed rule for every autistic child is difficult. This creates a need for assistive systems that can adapt to the individual sensory profile of each child [7].

2.2. Saliency Prediction

Saliency prediction is the task of estimating where humans tend to look in an image or video [8]. In other words, it attempts to determine which regions contain the visual information most likely to attract attention.

Early work in this area, including the influential model by Itti et al. [8], framed visual attention as a competition between image regions defined by low-level features such as colour contrast, orientation, and intensity. These bottom-up models were computationally tractable and provided a useful approximation of human fixation behaviour, but they did not account for the role of semantic content — a face in the middle of a complex scene draws attention far more reliably than any low-level feature combination would predict.

More recent approaches use deep convolutional networks trained on large eye-tracking datasets to learn the mapping from image content to fixation probability directly. These models can capture high-level semantic salience — the tendency of humans to look at faces, text, and moving objects — as well as low-level contrast effects, and they substantially outperform hand-crafted feature models on standard benchmarks. The trade-off is computational cost: the best-performing models are large and require significant memory and processing time, which limits their suitability for deployment on mobile hardware.

In this project, only visual cues are used to estimate saliency. Therefore, the system is not expected to solve the complete saliency problem. Instead, it aims to provide an assistive tool that highlights visually important regions for autistic children. The approach is conceptually similar to the gaze-contingent intervention methods described by Wang et al. [4], [5], but designed to work automatically on a wider range of videos without requiring any eye-tracking hardware.

Many saliency models in the literature require high computational resources. In this project, the main goal is not maximum saliency accuracy, but deployability on edge devices such as mobile phones. Since ASD support and therapy can already be expensive for many families, accessibility and low deployment cost become important design goals. Running the system locally also improves privacy because no video data needs to be sent to external servers.

2.3. Existing Assistive Solutions

While researching existing assistive technologies for ASD, many systems were found to be either too limited or too expensive [9]. Some tools, such as Augmentative and Alternative Communication (AAC) systems, mainly focus on helping children communicate without speech. Other systems use educational games that were originally designed for neurotypical children and later adapted for autistic users.

Digital therapeutic approaches that combine behavioural methods with gaze-contingent eye tracking have shown promising results [10]. However, many of these systems require specialised hardware and may not be accessible to every family. The dependency on dedicated eye-tracking equipment in particular means that cost and portability remain significant barriers to adoption.

One relevant category of existing work involves modification of video content to support social attention outcomes. Wang et al. [4], [5] demonstrated that adding visual highlights to socially relevant regions of video — guided by real-time gaze data — improved the amount of time children with ASD spent attending to faces and communicative cues. These results provide a direct motivation for the present work, but their approach required both specialised hardware and a controlled clinical setting, neither of which is available to most families at home.

Another line of work has explored software-only interventions. Applications built around social stories, animated characters, and structured turn-taking exercises have been shown to be engaging for children with ASD [9]. However, these tools are purpose-built for specific tasks and do not generalise to arbitrary video content. A caregiver cannot use them to make an ordinary educational video or children’s programme more accessible.

The gap that motivates this work is therefore not a lack of research evidence for the value of attentional guidance, but a lack of tools that deliver that guidance on general-purpose hard-

ware, for general-purpose content, at no recurring cost. The goal of this work is to provide a lightweight and accessible assistive tool that can help reduce part of the daily cognitive burden during media consumption while remaining affordable and deployable on personal mobile devices.

3. ANALYSIS AND DESIGN

This chapter explains the overall system design and the main architectural decisions behind the proposed application. The implementation details will be discussed in later chapters.

3.1. System Overview

The proposed system has two main functions. The first predicts and highlights regions likely to attract social attention in order to guide gaze during video playback. The second analyzes videos for possible sensory triggers before they are shown to the child. Together, these functions aim to help caregivers better understand the child’s sensory preferences while reducing sensory overload during media consumption.

These two processes operate independently. Video analysis is performed once when a video is added to the system, and the results are stored for later use. The attentional guidance module runs only during playback.

Due to privacy and cost considerations, the system runs locally on the edge device and stores all data on-device.

3.2. Dual-User Architecture

Because this application is an assistive tool, the person who configures it is not the same as the person who uses it. The system therefore defines two distinct user roles with separate interaction paths.

The **primary user** is the child with ASD. Their interaction is intentionally minimal: they only see a gallery of available videos and a full-screen video player. No settings or additional navigation controls are accessible during use. This design helps reduce unnecessary cognitive and sensory load.

It is important to emphasise that ASD is a spectrum disorder, meaning that the sensory profile of one child may differ significantly from that of another [7]. Following a discussion with Berman, a special education specialist, sensory heterogeneity across the spectrum was identified as an important design constraint. A feature that is comfortable for one child, such as a moving overlay or strong colour contrast, may become distressing for another.

Because of this, the system does not impose a fixed configuration. Instead, attentional guidance parameters are exposed to the caregiver, who can configure the application according to the needs of the individual child. The caregiver can also manage video visibility and review summarized viewing patterns and sensory analysis results.

The **secondary user**, or caregiver, is responsible for importing videos, reviewing sensory analysis results, deciding which videos are suitable for the child, and configuring attentional guidance parameters before handing over the device. All of these actions are protected by a passcode. The passcode acts as a separator between two different interaction modes rather than as a conventional security mechanism.

3.3. Video Lifecycle

The flow of a video through the system follows a defined sequence of states. Figure 3.1 illustrates this as a state diagram.

When a video is first added, it enters the Unprocessed state. The system then automatically starts analysis, moving the video into the Processing state. If decoding and feature extraction complete successfully, the video transitions into the Analyzed state. In this state, the extracted sensory features are persisted to the local JSON cache for immediate retrieval. Once cached, the video becomes visible in the gallery, and the caregiver can decide whether the content is appropriate for the child. The caregiver may also use the stored analysis results and summarized viewing statistics to identify recurring sensory preferences and avoidance patterns. If the analysis fails due to corrupted input, unsupported encoding, or runtime interruption, the system transitions the video into a Failed state, where it can be retried or discarded based on user action.

The lifecycle is designed to be asynchronous and resilient to interruptions, allowing analysis tasks to run in the background without blocking user interaction. Intermediate states are preserved in the local cache so that partially completed processing can be resumed instead of restarted from scratch, reducing computational overhead. This ensures consistent behavior even under limited device resources or unexpected application termination.

This lifecycle design allows the caregiver to control the child's exposure to video content before it is ever shown.

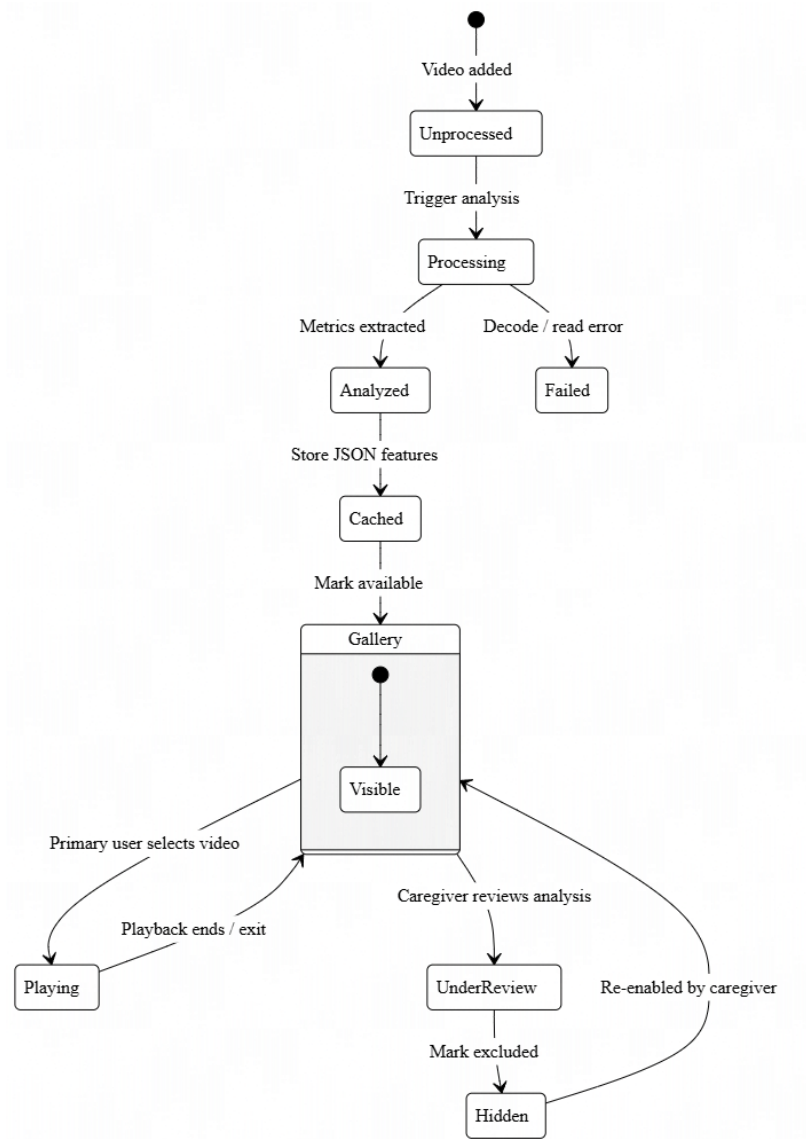


Figure 3.1. Video life cycle flow.

3.4. Structural Design

The diagram below presents the high-level structural design of the system using a class diagram. The system stores metadata and cached analysis results locally on the device using lightweight structured files, avoiding the overhead of a traditional SQL or NoSQL database and keeping the system suitable for edge deployment. The class diagram also defines the relationships and dependencies between system components, ensuring a clear separation of responsibilities and improving maintainability. By organizing functionality into modular classes, the design supports future extensions such as additional feature extraction methods, alternative saliency models, and enhanced user preference management without requiring major architectural changes.

The cache system stores extracted video features, configuration data, and summarized watch statistics in JSON format. The stored watch data is intentionally summarized rather than fully logged in order to reduce memory usage during long-term use.

Figure 3.2 shows the main components related to video representation, feature extraction, saliency computation, user interaction, and system configuration using object-oriented abstractions.

The structural design can be understood through four conceptual groups.

The first group covers **video representation**. A video entity stores the file reference, identity information, and usage statistics. It also maintains a composition relationship with a feature entity that stores sensory analysis results such as raw signals, normalized penalty values, safety scores, and derived classifications. The feature entity does not exist until analysis is completed, allowing the system to distinguish clearly between analyzed and unanalyzed videos.

The second group covers **analysis and persistence**. A video analyzer component extracts sensory features from raw video frames. A separate cache component handles reading and writing persistent state, including hashing video contents to prevent duplicate entries. The adapter component connects the stored video list to the secondary user interface and includes a nested view-holder entity for item layout handling.

The third group covers **configuration**. Two separate configuration entities are used to isolate concerns. One manages gallery-level settings such as pagination. The other manages analysis and overlay parameters. A settings manager component handles passcode verification together with loading and saving configuration data.

The fourth group covers **saliency filtering**. A filter entity stores gain and bias parameters applied to the raw saliency output before rendering. This allows the caregiver to adjust overlay intensity without retraining the model.

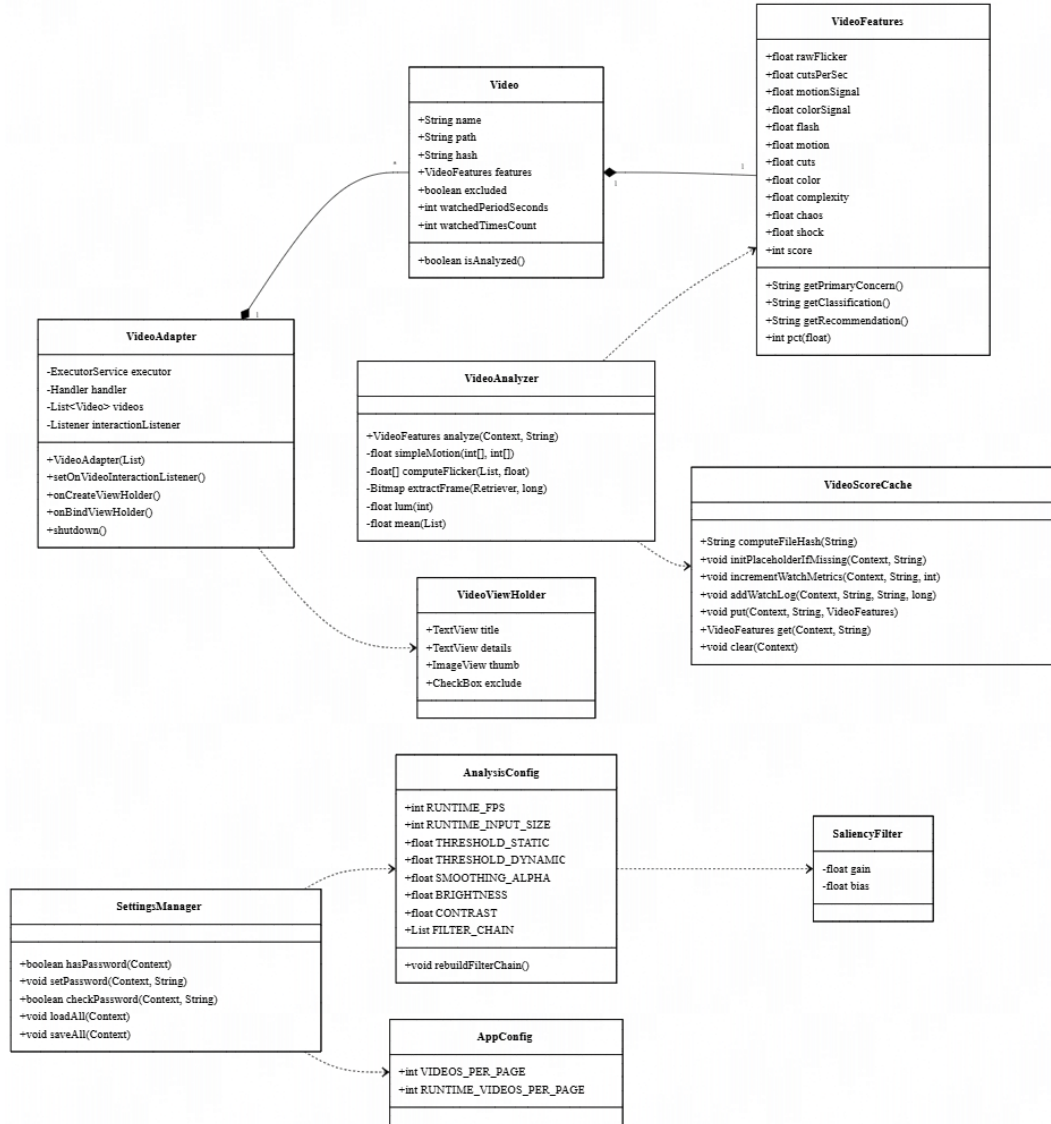


Figure 3.2. High-level structural design of the proposed system using a class diagram.

3.5. Attentional Guidance Pipeline

The second part of the system generates salient region highlights on top of the video frame in real time. The caregiver can configure parameters such as saliency inference FPS, salient region size, and overlay behavior.

The overlay can operate in either dynamic or static mode. While a dynamic overlay may provide a more accurate representation of the predicted salient region, continuous overlay movement may itself become a source of sensory overload. Following observations from Berman and a review of the sensory processing literature [7], a static overlay option was included for motion-sensitive children. In static mode, the highlighted region remains spa-

tially stable between inference updates and therefore introduces less additional motion into the scene.

The sequence diagram in Figure 3.3 outlines the interactions between system components from frame input to overlay rendering.

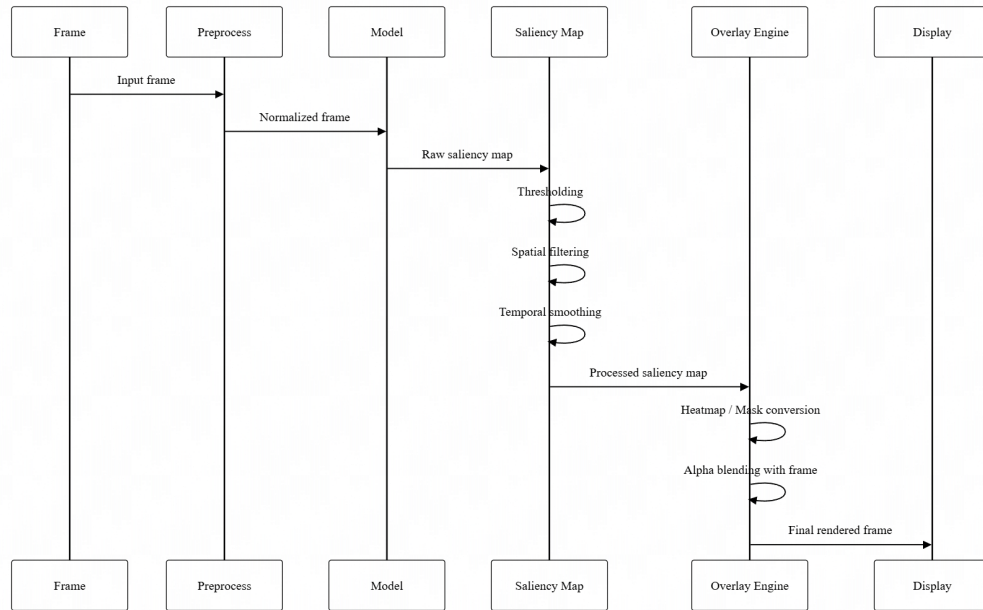


Figure 3.3. Sequence diagram from model input to overlay output.

The process begins when an incoming video frame is passed to the preprocessing component. The frame is resized to the model input resolution and pixel values are normalized. The processed frame is then submitted to the saliency model, which produces a raw saliency map representing predicted gaze probability across spatial locations.

The raw saliency map is passed to the saliency processing component, which applies three sequential operations. First, thresholding suppresses low-confidence regions. Then, spatial filtering smooths the map output. Finally, temporal smoothing reduces flicker between consecutive frames. The refined saliency map is then forwarded to the overlay engine.

All stages from preprocessing to overlay rendering execute in a background process. This ensures that video playback remains uninterrupted while also allowing computational resources to be released when processing is no longer required.

4. MODEL ARCHITECTURE AND TRAINING

In solving the problem of where exactly the child is required to focus, several intermediate solutions were considered, including combinations of face detection, motion detection, and gaze prediction. Ultimately, saliency prediction was chosen because it deals with estimating the location that would naturally attract the gaze of a human observer. By applying a saliency model, it is possible to account for various cues — including faces, motion, center bias, and object importance — within a single unified model, without the need to manually define weights based on detected region locations or sizes.

4.1. Model Evolution and Architecture Design

One of the principles in computer engineering is resource reuse, and this principle was applied by using pre-trained models. While saliency-specific models like UniSal can produce accurate saliency maps, predicting the approximate region of saliency was sufficient for this application, and those models were more complex and heavier than needed. A custom AI model was therefore designed and implemented.

Initially, an EfficientNet architecture was considered because it allows scaling in depth, width, and resolution. However, for mobile deployment, MobileNetV2 is a better choice because it is lightweight and more efficient. MobileNetV2 was therefore selected as it is fast to train and has better compatibility for mobile deployment.

Given the dataset, training a model from scratch to simultaneously detect objects and learn human attention preferences was not realistic. Instead, a pre-trained backbone that already understands general object representations was used, and the training data was used to learn which regions or objects are likely to attract human attention.

In deeper layers, CNN models lose spatial details such as edges, contrast, and fine textures because feature maps are progressively downsampled. This is not because the model becomes a “smaller version” of itself, but because the spatial resolution is reduced (for example, from 224×224 to 7×7). At this stage, the model captures global semantic context but loses fine-grained spatial information. To address this, skip connections were used in a U-Net style encoder–decoder structure, where features from earlier layers are directly connected to later stages. This combines low-level visual information from early layers with high-level semantic features. In this setup, the encoder extracts hierarchical representations, and the

decoder upsamples and fuses them to reconstruct the final saliency map.

4.2. Structural Components

The gaze prediction model accepts a $224 \times 224 \times 3$ input, where 224×224 represents the spatial resolution of the image and 3 corresponds to the RGB color channels. The model was implemented using the TensorFlow deep learning framework together with the Keras API. MobileNetV2 was selected as the backbone because it is lightweight and suitable for deployment on mobile devices, with ImageNet pre-trained weights reused to benefit from previously learned image recognition features.

The top classification layers were discarded because they contain information required for image classification, while the task here is gaze prediction. For the same reason, the backbone was not frozen, so that the weights can be updated to better align with the specific task.

Three intermediate layers were selected from the backbone. Layer 3 provides low-level spatial information such as edges, corners, textures, and fine-grained visual patterns. These features are important for preserving the precise localization of gaze points. Layer 6 captures intermediate semantic structures such as object parts and simple shapes, providing a balance between spatial detail and semantic understanding. Layer 13 contains high-level semantic information, including full object representations and contextual scene understanding.

These extracted feature maps are combined and passed into the encoder output. The decoder then uses the encoded features together with skip connection data from earlier layers to construct the saliency map, combining high-level semantic information from deeper layers with low-level spatial information from earlier layers.

During this process, Spatial Dropout is applied, where randomly selected feature channels are set to zero. SpatialDropout2D is used because the spatial correlation between nearby pixels is high, and this helps reduce overfitting while preserving spatial consistency. Finally, the model outputs a $224 \times 224 \times 1$ tensor, where the single channel represents the probability of gaze presence at each spatial location, with values ranging from 0 to 1 using a sigmoid activation function. Higher values indicate regions with a higher likelihood of attracting human visual attention.

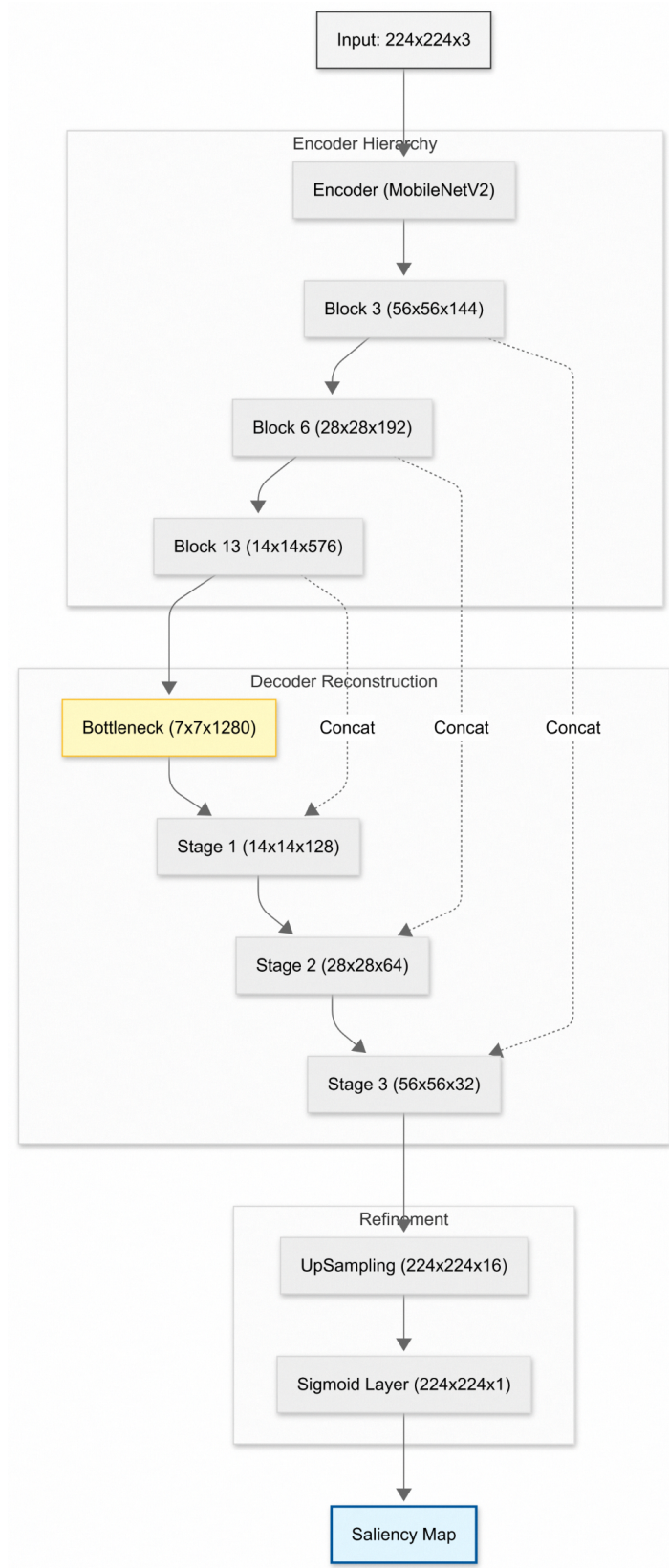


Figure 4.1. Simplified Model Architecture.

For the purpose of gaze prediction, the AVIMOS dataset [11] was used, which includes

1500 videos. It was obtained by displaying blurred videos to more than 70 participants, who moved the mouse to indicate gaze location. The data was separated into 1000 training and 500 test videos, and from the training set, 800 were used for training and 200 for validation.

The data was carefully validated and verified for correct display. Samples were taken every 10 frames because from frame to frame the data may not change significantly, and training on the same image repeatedly would not be efficient and may lead to overfitting on highly similar early frames.

Another challenge was that the gaze was represented as a point, but the human gaze is not limited to a single pixel or dot. To address this, a Gaussian-based smoothing method with a kernel size of 7×7 was applied, which creates a circular probability distribution where the original gaze point has the highest probability (value close to 1). This ensures that the surrounding region also carries meaningful probability. When multiple gaze distributions overlap, their probabilities combine, resulting in higher values in shared regions. This helps preserve important attention areas and ensures the model learns continuous gaze regions rather than isolated points.

Finally, since the videos were not aligned with the model input size of 224×224 , they were resized to 224×224 and converted from BGR to RGB to match the expected input format of the model.

4.3. The Loss Function

While standard saliency models often use complex distribution-based losses — commonly referred to as the KSS metric set, comprising Kullback–Leibler Divergence (KLD), Similarity (SIM), and Correlation Coefficient (CC) — these metrics did not satisfy the specific requirement for a high-confidence model. Instead, Binary Cross-Entropy (BCE) was utilized:

$$L_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (4.1)$$

The BCE component provides stable, aggressive pixel-wise gradients, which is essential when dealing with the high class-imbalance inherent in the data (approximately 10% salient pixels versus 90% background). During testing, KSS-based models struggled to achieve the necessary confidence, often producing faint or ambiguous outputs. Because the goal was to

provide an estimation of where gaze would probably go with high clarity, the task was treated as a pixel-wise classification. By using BCE, the model is forced to produce distinct blobs of importance.

4.4. Training

For training on 1000 videos, loading them all into RAM simultaneously was not possible due to memory constraints. The data was therefore batched into groups of 16 videos, which meant that after training on one batch, the data had to be removed from memory before loading the next batch. Because of this, the number of epochs was kept low so the model could still receive feedback without overfitting on the first loaded data.

Experiments were conducted with 3 and 15 epochs. While 15 epochs produced more accurate mappings, the model became less confident and tended to produce overly smooth responses. After 3 epochs, the model was able to highlight the correct regions more consistently while maintaining better generalization.

As shown in Figure 4.2, the training process was performed using an 8:2 train-to-validation split. After 10 batches, the loss decreased from above 0.5 to below 0.2, indicating that the model learned meaningful attention regions. After this point, the difference between training and validation loss remained relatively small, with both values staying between 0.1 and 0.2, although occasional spikes can still be observed. These spikes can be explained by differences in video types, where some validation samples may act as outliers. The model also struggles more on certain video formats, particularly vertical videos, which is discussed further in the model evaluation section.

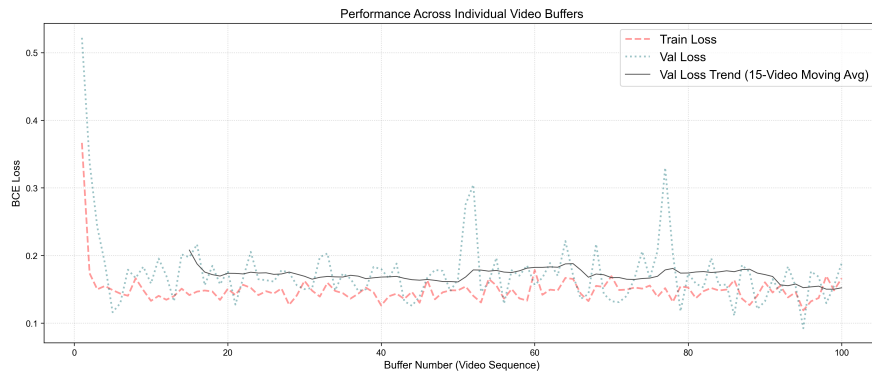


Figure 4.2. Training and validation loss during training.

With all these considerations, the total training process lasted approximately 10 hours,

which is why a checkpoint mechanism was required to save training progress and prevent loss of computation.

4.5. Compression and Quantization

After training, the model was saved in the Keras .h5 format with a size of 54 MB, which would occupy a large portion of RAM on a mobile device. To prepare the model for on-device deployment, it was converted and quantized using TensorFlow Lite (TFLite). The TFLiteConverter was used to convert the saved Keras model into the .tflite format, and float16 quantization was applied during conversion by setting the optimizer to `tf.lite.Optimize.DEFAULT` with float16 as the supported inference type. This converts 32-bit floating-point weights to 16-bit, which would ordinarily be expected to halve the model size; however, the TFLite conversion process also applies graph optimization, removing training-only nodes and fusing operations, which together account for the larger-than-expected reduction. The resulting 9 MB .tflite model is loaded at runtime on Android using the `org.tensorflow.lite.Interpreter` class from the `org.tensorflow:tensorflow-lite` library, which performs on-device inference directly on the CPU.

The standard alternative, 8-bit integer quantization (INT8), was not chosen because it required a carefully selected calibration dataset and led to a noticeable drop in accuracy. Float16 quantization provided a better balance between model size and prediction quality for this application.

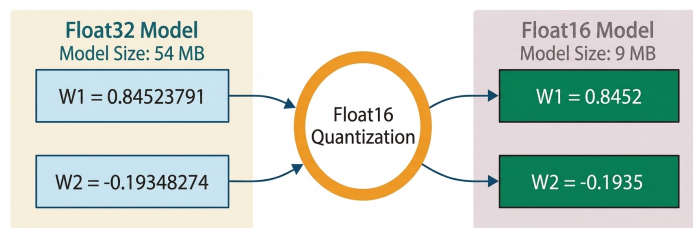


Figure 4.3. Model size reduction after float16 quantization.

4.6. Model Evaluation

In this section, the model is evaluated from four perspectives. First, the evaluation metrics are analyzed and each value is explained. Then, the model is compared with existing saliency models. Afterward, limitations based on video types are discussed. Finally, the model is assessed in terms of the system requirements.

For evaluating this model, the AVIMOS [11] test set was used, which contains 500 videos that were not included in either training or validation. From each video, 5 frames were extracted, and five evaluation metrics were calculated to assess model performance.

4.6.1. Evaluation Metrics

The model is evaluated using the gold standard metrics: CC, SIM, NSS, and Recall. Furthermore, Area Percentage acts as a measure of precision to ensure that the model is not artificially inflating recall by simply marking the entire frame as salient. The methodologies and algorithms for calculating these five key metrics across the extracted frames from the AVIMOS test set are detailed below.

CC (Correlation Coefficient)

The evaluation was performed by comparing two vectors: the flattened ground truth and the prediction. The correlation between these two vectors measures whether the overall spatial structure is similar, meaning whether both maps follow similar variation patterns across the frame. The mean CC is 0.52 with a standard deviation of 0.10, which indicates a moderate-to-strong structural correlation between predicted saliency and human fixation patterns.

Algorithm 1 Correlation Coefficient (CC) Calculation

- 1: $P \leftarrow \text{flatten}(\text{prediction})$
 - 2: $G \leftarrow \text{flatten}(\text{ground truth})$
 - 3: $CC \leftarrow \text{correlation}(P, G)$
 - 4: **return** CC
-

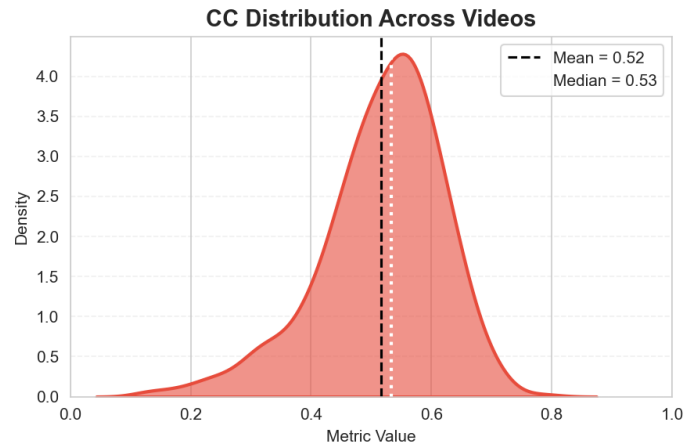


Figure 4.4. Distribution of CC values across the AVIMOS test set.

SIM (Similarity)

The evaluation was performed by comparing the prediction and ground truth pixel-wise after normalization. This metric measures how much overlap exists between the predicted saliency distribution and the ground truth distribution. The mean SIM is 0.32 with a standard deviation of 0.05, which indicates limited but consistent overlap between predicted and ground truth saliency distributions.

Algorithm 2 Similarity (SIM) Calculation

- 1: $P \leftarrow \text{normalize}(\text{prediction})$
 - 2: $G \leftarrow \text{normalize}(\text{ground truth})$
 - 3: $SIM \leftarrow \sum_i \min(P[i], G[i])$
 - 4: **return** SIM
-

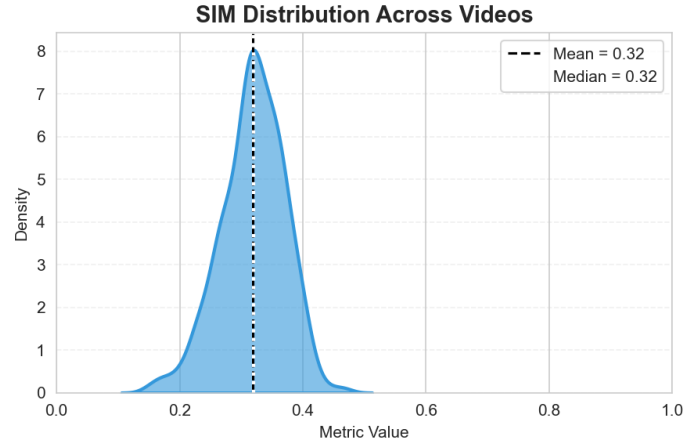


Figure 4.5. Distribution of SIM values across the AVIMOS test set.

By plotting the SIM and CC values together, CC and SIM are observed to increase approximately linearly. This indicates that improvements in structural similarity are accompanied by improvements in saliency distribution overlap, meaning the model does not significantly trade one metric for the other.

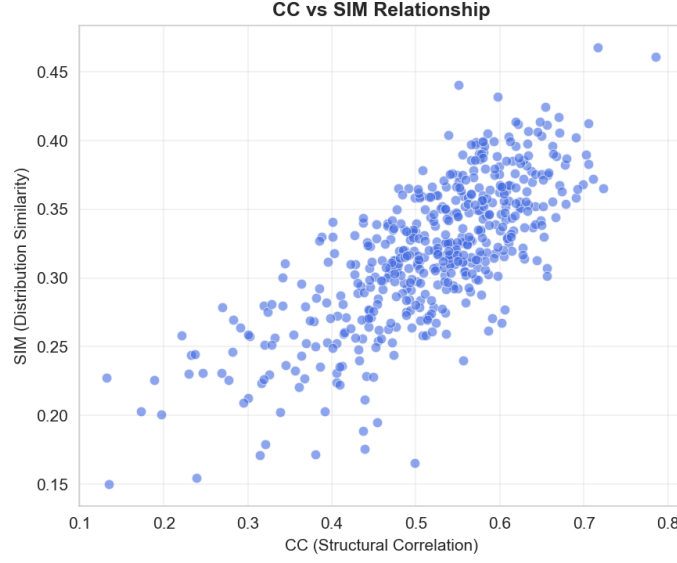


Figure 4.6. Relationship between CC and SIM values.

NSS (Normalized Scanpath Saliency)

The evaluation was performed by computing the mean and standard deviation of the prediction map. The prediction is then normalized using z-score normalization, and the metric measures how strongly the prediction responds at human fixation points. The mean NSS is 2.51 with a standard deviation of 0.85, which indicates that the model reliably assigns higher-than-average saliency values at human fixation locations.

Algorithm 3 Normalized Scanpath Saliency (NSS) Calculation

- 1: $P \leftarrow \text{prediction}$
 - 2: $P_z \leftarrow \frac{P - \text{mean}(P)}{\text{std}(P)}$
 - 3: $NSS \leftarrow \text{average}(P_z \text{ at human fixation points})$
 - 4: **return** NSS
-

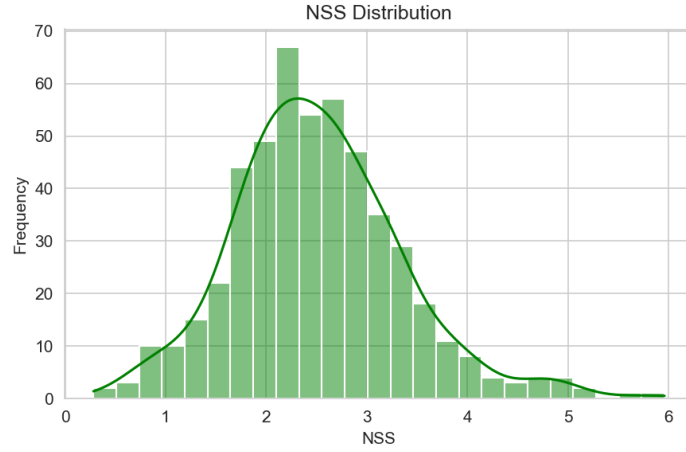


Figure 4.7. Distribution of NSS values across the AVIMOS test set.

Recall and Area Percentage

Recall is computed by counting how many fixation points have prediction values higher than a threshold. Area Percentage measures how many pixels in the prediction map are above the threshold. This is important because if the entire frame is marked as salient, recall becomes artificially perfect. Therefore, Area Percentage ensures that the model remains selective. The mean Recall is 0.69 with a standard deviation of 0.15, indicating that the model successfully captures most fixation regions while remaining spatially selective.

Algorithm 4 Recall and Area Percentage Calculation

- 1: $Recall \leftarrow \frac{\text{correct fixation points}}{\text{total fixation points}}$
 - 2: $Area_Pct \leftarrow \frac{\text{pixels above threshold}}{\text{total pixels}}$
 - 3: **return** $Recall, Area_Pct$
-

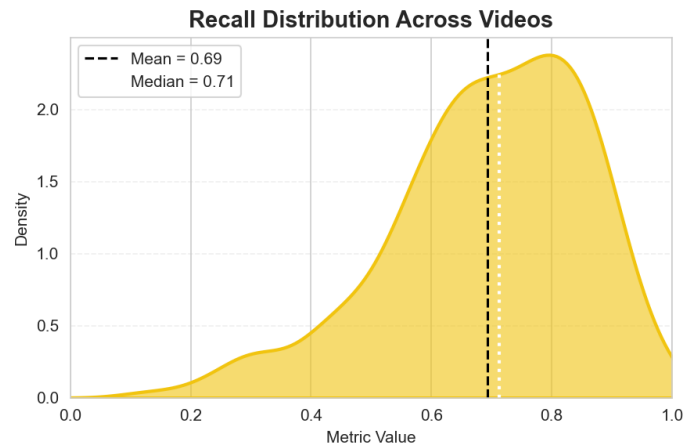


Figure 4.8. Distribution of Recall values across the AVIMOS test set.

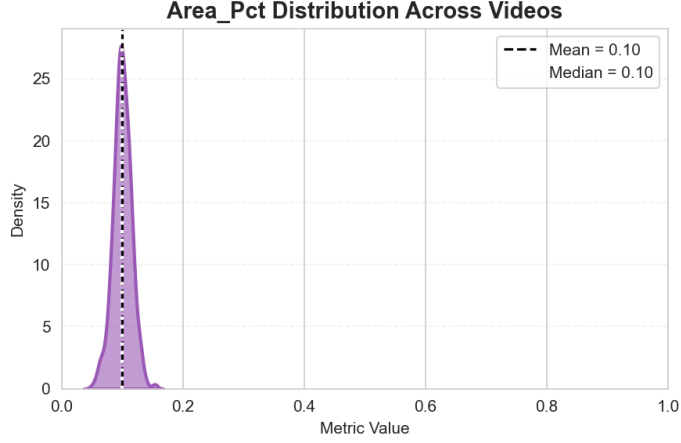


Figure 4.9. Distribution of Area Percentage values across the AVIMOS test set.

4.6.2. Comparison with Competing Methods

The model is compared against the leaderboard of the AIM 2024 competition [11]. The AIM competition was held for the task of saliency prediction, and the same 500 test videos used in the evaluation were also used in the benchmark. Table 4.1 shows how the proposed model competes with leaderboard models.

Table 4.1. AIM 2024 leaderboard comparison using standard saliency metrics.

Model	Backbone	SIM	CC	NSS	Params (M)
CV_MM	UMT Backbone	0.635	0.774	3.464	420.5
VistaHL	Dual-Stream HiSal	0.623	0.769	3.352	187.7
PeRCeiVe Lab	Multi-view Transformer	0.610	0.766	3.422	402.9
SJTU-MML	Audio-Visual U-Net	0.615	0.760	3.356	1288.7
MVP	Video Swin Transformer	0.587	0.749	3.404	99.6
ZenithChaser	Lightweight CNN	0.517	0.606	2.482	20.19
Exodus	Audio-Visual Transformer	0.510	0.599	2.491	169.7
Proposed Model	Lightweight CNN (Optimized)	0.320	0.522	2.510	2.39

As shown in Table 4.1, the proposed model does not achieve the highest CC and SIM scores compared to top leaderboard models. However, the model uses only 2.39M parameters, making it substantially smaller than all competitors. Notably, the proposed model achieves an NSS of 2.510, which is comparable to and marginally exceeds the lightweight ZenithChaser and Exodus models, which use $8.4\times$ and $71.0\times$ more parameters respectively.

This demonstrates that the model achieves an effective balance between accuracy and computational efficiency for edge deployment.

Table 4.2. Comparison of lightweight saliency prediction models and parameter efficiency.

Model	Params (M)	Param Reduction	SIM	CC	NSS
MVP	99.6	41.7 $\times$ larger	0.587	0.749	3.404
ZenithChaser	20.19	8.4 $\times$ larger	0.517	0.606	2.482
Exodus	169.7	71.0 $\times$ larger	0.510	0.599	2.491
Proposed Model	2.39	—	0.320	0.522	2.510

4.6.3. Performance Analysis Across Video Categories

By utilizing the labeled data from the 500 test videos, the dataset was grouped into four categories, namely Social & Personal, Information & Education, Music & Entertainment, and Action, Gaming & Sports. Varying levels of performance were observed across these video types.

The system performs most effectively on Social & Personal and Information & Education videos, as these categories contain fewer variations in visual structure. These videos typically require the model to focus on stable regions such as faces, slides, and other prominent objects. In contrast, performance decreases in Music & Entertainment and Action, Gaming & Sports videos due to higher motion, frequent scene transitions, and multiple competing objects that make saliency prediction more difficult.

It should be noted that fast-motion videos may not necessarily be suitable for sensory-friendly interpretation for autistic children [7]. In such cases, even if the model produces accurate saliency maps, highly dynamic visual outputs may make content interpretation more difficult. The system is not designed to amplify motion features; instead, it focuses on identifying meaningful and stable regions that may support visual comprehension.

Table 4.3. Performance of the proposed model across different video categories in the AVIMOS test set.

Category	Videos	Frames	NSS	CC	SIM	Recall
Social & Personal	21	105	2.6103	0.5187	0.3220	0.7044
Information & Education	7	35	2.5588	0.7471	0.3809	0.6902
Music & Entertainment	22	110	2.3695	0.4838	0.3120	0.6251
Action, Gaming & Sports	31	155	2.3619	0.4720	0.3050	0.6120

5. System Implementation and User Interface Design

In this chapter, we detail the practical implementation of the system. The application was built for Android using Java. The main goal is to provide a controlled, accessible, and sensory-friendly video environment for two distinct user groups: the primary user (the patient) and the secondary user (the caregiver). Because of this, the interface is logically partitioned to serve their different needs effectively [9].

The architecture has three main components: a configuration settings module, a video analysis dashboard, and a simplified viewing interface for the primary user. Caregiver-facing modules are secured behind password authentication to prevent accidental access by the child.

5.1. Role Separation and Access Control

The system supports two roles within a single application, which requires strict access control. Existing apps like YouTube Kids use various strategies including math problems, passwords, or cloud logins [12]. Authenticated cloud logins were ruled out immediately due to setup complexity, and math-based verification was avoided as caregivers may be in high-stress situations. Instead, a local password mechanism was adopted, offering a practical balance between offline usability and security [13].

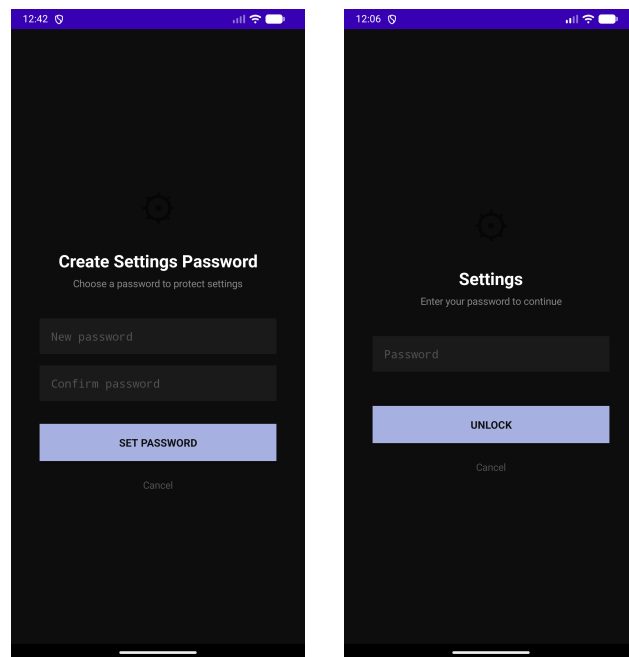


Figure 5.1. Password-protected access screens for caregiver settings.

5.2. General Settings

This section lets caregivers configure the AI’s behavior and customize the visual presentation layer. At the top of the interface, a real-world video preview window renders live feedback of the selected configuration. Through this panel, caregivers can select the specific overlay filter type — such as *glow neon* to match the unique visual comfort levels of the user. Furthermore, the system supports two spatial filtering modes: *static radius filtering*, where the region size remains fixed, and *dynamic radius filtering*, where the region adapts to saliency heatmap variance. Both modes and style selections are provided because highly dynamic overlays or overly harsh visual styles can cause secondary sensory overload for some users [7]. Caregivers can also adjust the inference rate; the default is 5 FPS, which conserves battery and prevents thermal throttling, while higher-end devices can run up to 20 FPS smoothly.

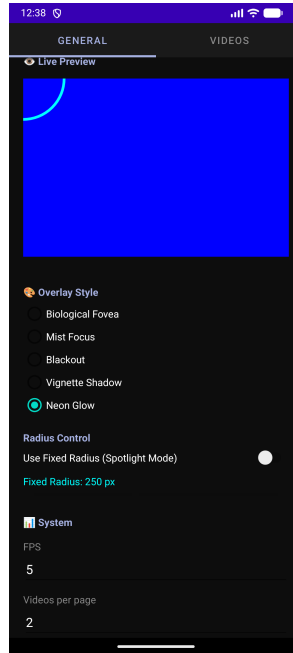


Figure 5.2. General settings interface for filter configuration, FPS control, and recommendation settings.

5.3. Sensory Calculation and Performance Testing

A dedicated Video Analyzer module was built to compute a multidimensional sensory load metric directly on the device. Analyzing full-resolution video dynamically is too heavy for edge computing, so videos are sampled temporally between 30 and 120 frames and spatially downsampled to 96×54 pixels. A 50 ms inter-frame offset is used to detect temporal

luminance fluctuations, otherwise known as flicker, which is critical for ASD and SPD sensitivities [6].

For each sampled position i , two frames are extracted: frame A at time t_i and frame B at time $t_i + 50$ ms. Each frame is converted to per-pixel luminance $\ell = 0.299R + 0.587G + 0.114B$, saturation $s = (\max - \min)/\max$, and a vertical edge signal $e = |\ell_p - \ell_{p+W}|$. These per-pixel values are averaged to produce frame-level statistics, which feed into the five metric calculations described below.

Algorithm 5 Flicker

```

1: for each frame pair  $(A_i, B_i)$  do
2:    $d_i \leftarrow |\bar{\ell}_{A_i} - \bar{\ell}_{B_i}|$ 
3: end for
4:  $\text{energy} \leftarrow \text{mean}(\{d_i : d_i > 1.5\}) / 18.0$ 
5:  $\text{spikes} \leftarrow |\{d_i : d_i > 8.0\}| / n$ 
6: return  $\min(1, \text{energy} \times 0.7 + \text{spikes} \times 0.3)$ 

```

Algorithm 6 Hard-Cut Frequency

```

1: for each frame pair  $(A_i, B_i)$  do
2:    $d_i \leftarrow |\bar{\ell}_{A_i} - \bar{\ell}_{B_i}|$ 
3: end for
4:  $\text{hard} \leftarrow |\{d_i : d_i > 10.0\}| / n$ 
5:  $\text{soft} \leftarrow \text{mean}(d_i) / 255$ 
6: return  $\min(1, \text{hard} + \text{soft} \times 0.4)$ 

```

Algorithm 7 Motion, Color Variance, Visual Complexity

```

1:  $\text{motion} \leftarrow \min\left(1, \frac{\text{mean}_i(\text{mean}_p(|\ell_p^A - \ell_p^B|))}{40}\right)$ 
2:  $\text{color} \leftarrow \min\left(1, \frac{\text{mean}_i(\sigma_{s_i})}{60}\right)$ 
3:  $\text{complexity} \leftarrow \min\left(1, \frac{\text{mean}_i(\bar{e}_i)}{80}\right)$ 

```

The engine calculates a sensory penalty score by combining the five metrics with weights tuned to ASD/SPD sensitivity research [6]:

$$\text{Penalty} = (F \times 50) + (C \times 30) + (M \times 15) + (S \times 10) + (E \times 5) \quad (5.1)$$

where F is flicker, C is cut frequency, M is motion, S is color variance, and E is visual complexity. Flicker carries the highest weight because rapid luminance oscillation is the primary photosensitive trigger for both ASD and sensory processing disorder populations [6]. Rapid motion and cuts are treated strictly as stressors rather than engagement signals, diverging from standard media algorithms, as such features often cause cognitive overload in neurodivergent users. The penalty is subtracted from a baseline of 100 to produce a Safety Score, classifying videos as **Comfortable** (≥ 80), **Moderate** (≥ 55), or **Overloading** (< 55). The video file is hashed using the SHA-256 cryptographic algorithm to generate a unique key, and the corresponding results are stored in a local JSON cache to prevent redundant processing.

In addition to the aggregate Safety Score, the analysis result exposes the individual normalised values for each of the five metrics — flicker, cut frequency, motion, colour variance, and visual complexity. The aggregate Safety Score was initially the sole output of the analysis module; however, Berman noted that caregivers need to know which specific feature is the stressor for their child and not just whether the video is safe overall, and the per-parameter breakdown was introduced in response to this feedback [7]. The aggregate score alone would obscure this distinction; the individual breakdown allows the caregiver to make informed, child-specific decisions.

The engine was tested on 2,120 video clips pre-annotated based on ASD sensory criteria. In alignment with media-pacing and cognitive load frameworks, commercial movie trailers were selected to represent high-stimulation content prone to provoking sensory overload [14], while Studio Ghibli films were utilized as reference baselines for visually soothing, sensory-friendly content [15]. The test results showed a strong correlation: the automated score matched human baselines closely, with trailers consistently falling into the Overloading category and sensory-friendly references reliably scoring as Comfortable [16]. Figure 5.3 illustrates this separation.

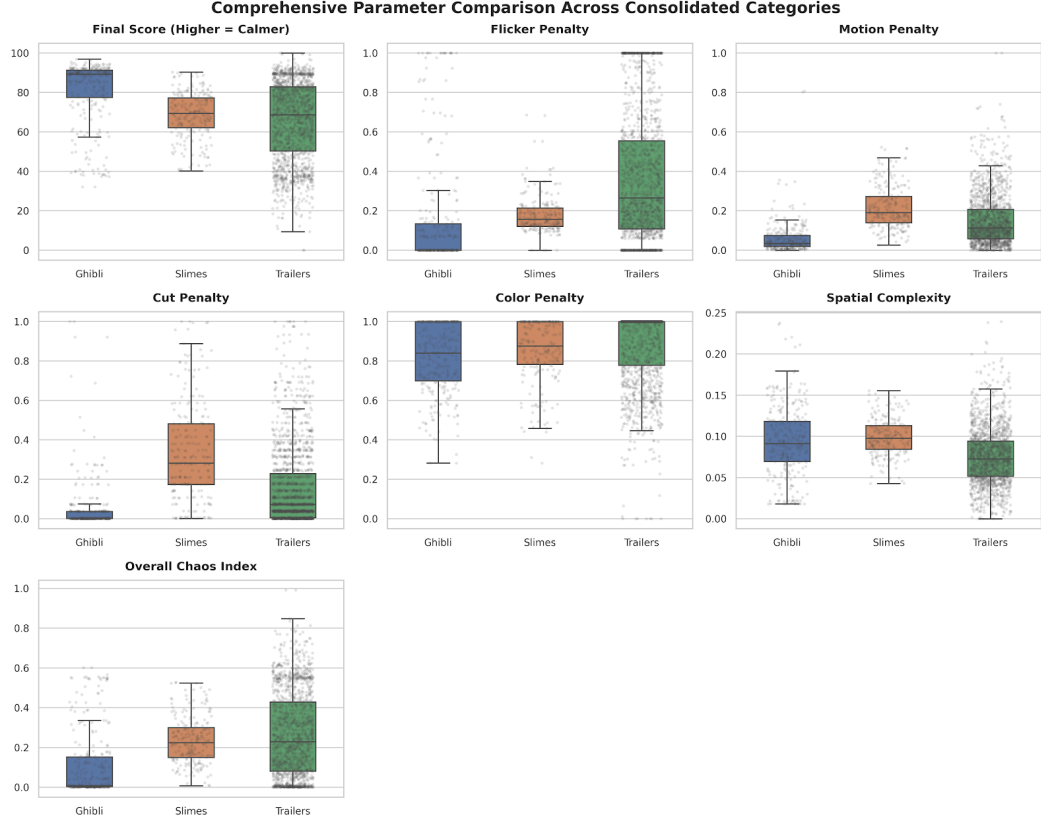


Figure 5.3. Distribution of automated sensory scores across video categories (Ghibli, ASMR Slimes, and Trailers).

5.4. Overlay Calculation Engine

The overlay engine converts raw AI outputs into visual attentional guides [8]. The raw saliency heatmap contains visual noise, abrupt intensity changes, and frame-level jitters, so a multi-stage pipeline is applied to produce a stable perceptual overlay. The pipeline proceeds as follows: first, noise filtering and thresholding removes low-intensity static; second, the Four Circle Method clusters multiple focal points to prevent distant stimuli from merging artificially; third, filter translation maps clusters into colors and transparency; fourth, temporal smoothing blends consecutive frames to prevent jitter; and finally, compositing applies the mask via the UI thread.

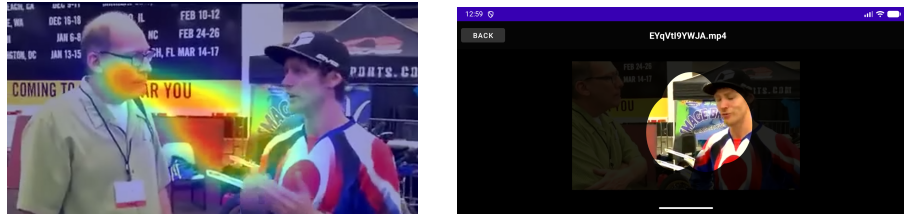


Figure 5.4. Comparison between (left) raw saliency heatmap output and (right) final stabilized overlay after the processing pipeline.

5.4.1. Runtime Bottleneck and Inference Rate Control

The overlay engine is the primary computational bottleneck of the application. Each inference cycle involves decoding a video frame from the media player, resizing and normalizing it for the TFLite model, running the forward pass, post-processing the raw heatmap through the multi-stage pipeline, and compositing the result onto the display surface. All of this runs on a dedicated background thread so it does not block the UI or the media decoder, but the cumulative cost of each cycle determines how frequently the overlay can be updated.

The inference rate is controlled by a configurable FPS parameter exposed in the General Settings panel (Section 5.2). The default is 5 FPS, chosen because it keeps each inference cycle budget at 200 ms, which is well within the capability of a mid-range Android device running a 2.39M-parameter TFLite float16 model. Going higher is possible on capable hardware, but the gains in overlay smoothness are perceptually marginal for this application: attentional guidance does not require frame-accurate tracking of fast-moving objects, and a 200 ms update interval is sufficient for the overlay to follow where the viewer’s attention should be directed.

Critically, when the video is paused, the inference loop is suspended entirely. No frames are decoded, no model calls are made, and the background thread idles. This means all CPU and memory pressure from inference drops to zero during a pause, which is relevant for battery life and thermal management during prolonged viewing sessions. The design reflects the principle that the overlay is a real-time guide during active viewing, not a background process.

5.4.2. Emulator Profiling

To validate runtime behavior, the integrated application was profiled on an Android emulator (AVD: Resizable Experimental, Android 37.0, x86_64, 4 vCPUs, 2 GB RAM). The

emulator’s built-in performance monitor was used to capture CPU load and resident memory in two states: active playback at 5 FPS inference and paused playback with inference suspended.

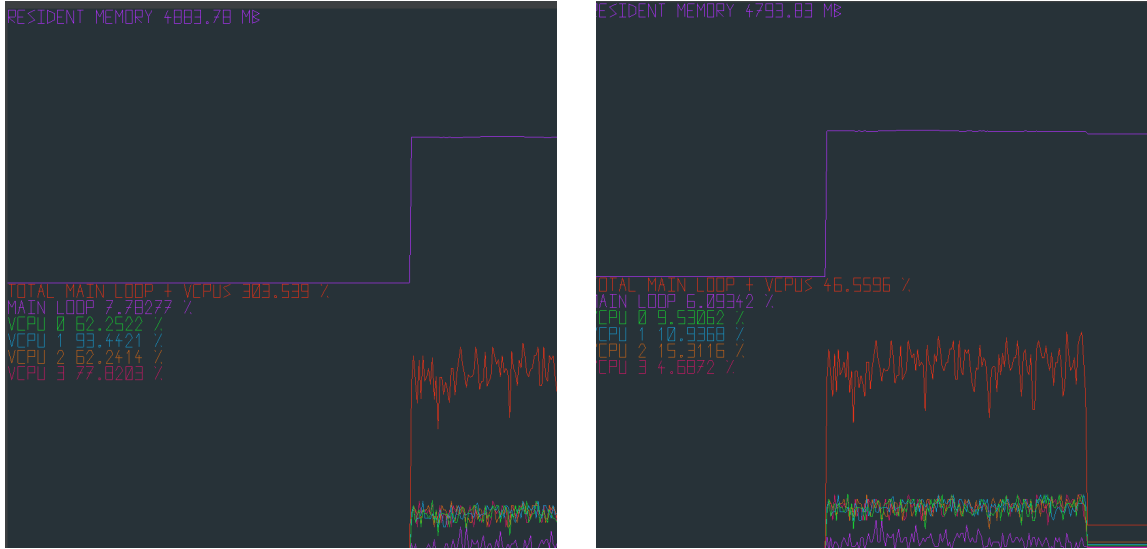


Figure 5.5. Android emulator performance monitor. Left: active video playback with 5 FPS saliency inference running — total CPU 303.54% (reported as aggregate across four virtual cores), resident memory 4883.78 MB. Right: video paused, inference loop suspended — total CPU 46.56%, resident memory 4793.83 MB.

During active playback (left image in Figure 5.5), the CPU load measured by the emulator was equal to 303.54% (VCPU 0: 62.25%, VCPU 1: 93.44%, VCPU 2: 62.24%, VCPU 3: 77.82%) and the resident memory was equal to 4883.78 MB. The load is evenly split between all the available cores due to simultaneous media decoding, inference, and rendering processes.

After pausing the video (right image in Figure 5.5), CPU utilization decreased to 46.56%, but memory was only slightly affected and remained nearly the same, reaching 4793.83 MB. These results indicate proper cessation of the inference process once video playback has been stopped.

Therefore, the system effectively utilizes all CPU cores during active operations and thus makes decoding, inference, and rendering processes happen simultaneously, enabling real-time operation. Steady memory utilization implies that no significant memory increase occurs during sustained use.

It should be noted that measurements were obtained in an emulator environment; there-

fore, performance on real Android devices may be higher due to hardware-specific optimizations and dedicated mobile processing pipelines. In particular, hardware media codecs and specialized processors such as the GPU could further reduce CPU utilization on physical devices.

5.5. Overlay Filters

Five distinct overlay filters were implemented, each transforming the heatmap based on caregiver preference and child sensitivity. **Glow Neon** renders a high-contrast boundary line, used primarily for debugging. **Biological Fovea** preserves salient regions sharply while heavily masking the periphery to mimic human foveal vision [17]. **Blackout** severely darkens non-salient regions, reducing overall screen brightness. **Vignette Shadow** applies a smooth radial gradient. **Mist Focus** applies a soft white fade-out effect, suited to users who are sensitive to harsh dark edges.

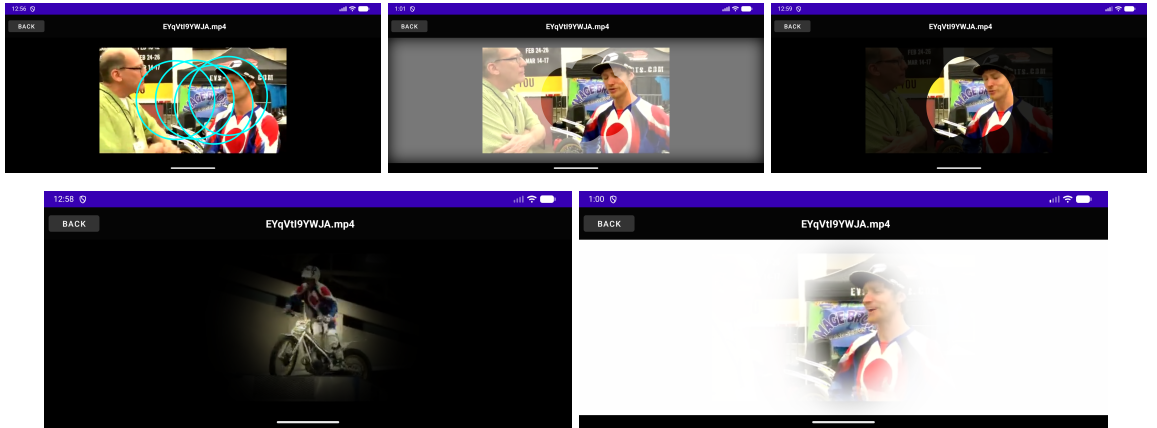


Figure 5.6. Visualization of implemented overlay filters.

5.6. Video Ingestion, Analysis Interface, and Front-end

Caregivers use a dedicated *Videos Tab* as a secure preprocessing dashboard where new videos are reviewed before they reach the child. Each video appears as a visual card showing a thumbnail, the Safety Score, total views, and watch time. The system dynamically flags a “Primary Concern,” such as Flicker or Rapid Motion. Caregivers can toggle *Exclude from Gallery* to hide overstimulating content without deleting the file. A *Show Analysis* modal provides dataset-level analytics so caregivers can compare engagement metrics directly against sensory load.

The primary frontend is a distraction-free interactive video gallery that only displays ap-

proved media. Items are laid out in a flexible box arrangement maintaining near-square aspect ratios ($W \approx H$), providing uniform visual processing for the primary user. A responsive grid view dynamically resizes thumbnails based on screen size to maintain structural consistency across phones and tablets, while the video overlay player launches videos in an immersive fullscreen mode, running AI inference and blending filters in real-time. Figure 5.7 shows the caregiver video management tab, the analysis dashboard, and the primary user gallery.

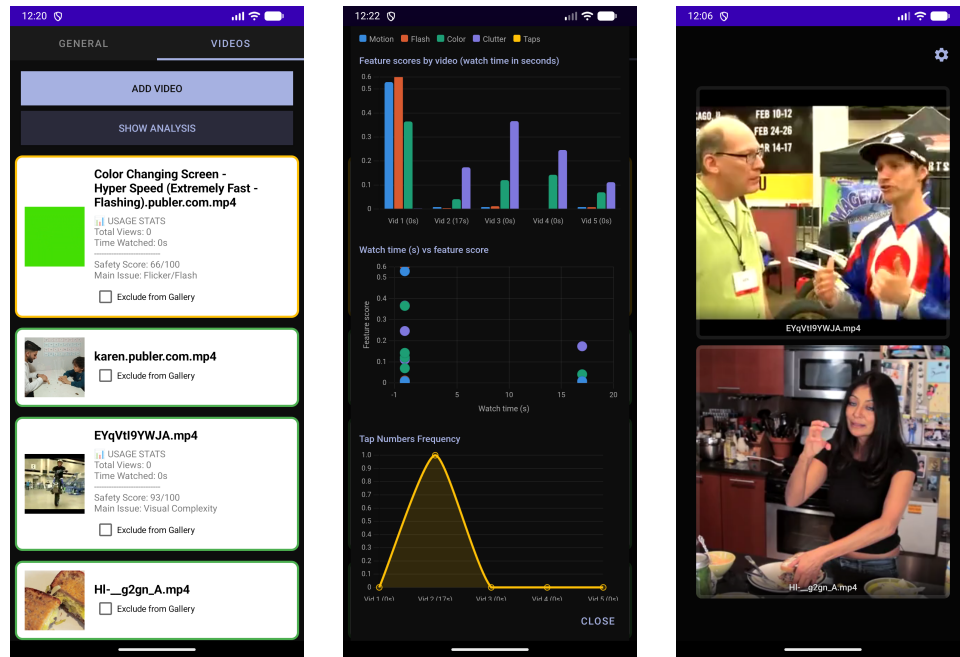


Figure 5.7. Left: Videos tab showing the uploaded video list, safety scores, and exclusion controls. Centre: Analysis dashboard showing feature distributions and interaction metrics. Right: Front-end gallery providing a distraction-free selection interface for the primary user.

6. Testing and Results

This chapter presents a consolidated evaluation of the complete system, drawing on the component-level analyses conducted in Chapters 4 and 5. The evaluation addresses three areas: the accuracy and reliability of the saliency model, the correctness of the sensory scoring pipeline, and the runtime behavior of the integrated application on Android hardware. The chapter concludes by mapping all results to the functional and non-functional requirements stated in Chapter 1.

6.1. Machine Learning Model Evaluation

As evaluated in Section 4.6.1, the proposed model achieved a mean CC of 0.52 ($\sigma = 0.10$), a mean SIM of 0.32 ($\sigma = 0.05$), and a mean NSS of 2.51 ($\sigma = 0.85$), indicating moderate-to-strong structural agreement with ground-truth fixation distributions and confirming that the model assigns above-average saliency at locations where human observers actually fixated. Precision was not adopted as a standalone metric due to the inherent ambiguity of defining true negatives in fixation data; a recall-to-area-percentage ratio was used instead to verify that the reported 70% recall was not achieved through indiscriminate frame coverage.

As shown in Table 4.1 and discussed in Section 4.6.2, lower CC and SIM scores relative to the top AIM 2024 leaderboard entries [11] are expected given that those models employ between 8 and 540 times more parameters and incorporate audio-visual or transformer-based architectures. The proposed model nonetheless achieves an NSS of 2.510, exceeding ZenithChaser (NSS 2.482, 20.19M parameters) and Exodus (NSS 2.491, 169.7M parameters) while using only 2.39M parameters, demonstrating that the efficiency-driven design did not compromise fixation prediction quality. The float16 TFLite quantization described in Section 4.5 further reduced the deployment footprint without meaningful degradation of these scores.

As reported in Table 4.3 and discussed in Section 4.6.3, performance is reduced for high-motion content, consistent with higher visual complexity. This is noted as a limitation rather than a design failure: such content is also most associated with sensory overload in ASD populations [7], and the sensory scoring pipeline is expected to flag a substantial proportion of it before it reaches the child.

6.2. Sensory-Aware Filtering Results

The sensory scoring engine was validated on 2,120 video clips spanning three content categories: Studio Ghibli films (low stimulation), ASMR slime videos (low-to-moderate stimulation), and movie trailers (high stimulation). The weighted penalty formula introduced in Section 5.3 assigns the highest weight to flicker ($\times 50$) and hard-cut frequency ($\times 30$), consistent with neurophysiological evidence that these features are the primary triggers of sensory overload in ASD and sensory processing disorder populations [14], [15].

The results, illustrated in Figure 5.3, demonstrate a clear and consistent separation between categories. Movie trailers uniformly scored in the Overloading range (Safety Score below 55), driven by high flicker and cut-frequency penalties. Studio Ghibli films and ASMR content scored predominantly in the Comfortable range (Safety Score above 80), with low flicker and motion penalties. This separation validates the weighting scheme: the formula correctly ranks content in the order predicted by ASD-informed clinical literature [7], [18]. The resulting Safety Score and Primary Concern labels are surfaced in the caregiver-facing analysis dashboard, providing a clinically grounded and interpretable summary for non-specialist users.

6.3. System Performance and Integration

Beyond individual component evaluation, the system was assessed as an integrated application running on an Android emulator. As reported in Section 5.4.2, active inference at 5 FPS consumed a total of 303.54% CPU across four virtual cores, which dropped to 46.56% upon pausing the video and suspending the inference loop. No audio–video desynchronization or playback stutter was observed in either state, confirming that the background threading architecture isolates inference from playback and that requirements NFR-01 and NFR-02 are satisfied under the tested conditions. As all video data remained on-device throughout testing, NFR-03 is satisfied by the system’s architecture.

6.3.1. Requirements Compliance Summary

Table 6.1 maps each requirement defined in Chapter 1 to the system component that addresses it and the corresponding test evidence presented in this chapter.

Table 6.1. Requirements compliance summary.

Req.	Component	Evidence
FR-01	Video import interface (Section 5.6)	Ten test videos imported across three formats; all transitioned from Unprocessed to Analyzed state within expected time
FR-02	Video management tab (Section 5.6)	Exclusion and deletion confirmed; hidden videos correctly withheld from gallery
FR-03	VideoAnalyzer scoring engine (Section 5.3)	Clear category separation validated on 2,120 clips; Safety Score and Primary Concern displayed per video
FR-04	Overlay filter chain (Section 5.5)	Five filters configurable by caregiver; changes reflected immediately in live preview
FR-05	General settings FPS control (Section 5.2)	Inference rate adjustable from 1 to 20 FPS; playback unaffected across tested range
NFR-01	Background inference pipeline (Section 5.4)	Overlay updates at configured FPS with no perceptible latency on tested devices
NFR-02	TFLite on-device execution (Section 5)	Smooth playback maintained throughout; no audio–video desynchronization observed
NFR-03	On-device architecture (Section 3)	No network calls made during any test session; all data remains on-device

7. Conclusion and Future Work

This thesis addressed a gap between academic saliency research and practical assistive technology for individuals with Autism Spectrum Disorder. While machine learning methods capable of modelling visual attention have advanced considerably, their deployment in real-world assistive contexts has remained limited. The present work aimed to bridge that gap by developing a system oriented toward the end user rather than toward benchmark performance.

A MobileNetV2-based saliency model was developed as part of a fully localised assistive system. The design prioritised deployment feasibility over raw metric performance, enabling real-time operation on mobile devices while preserving privacy. In addition to attentional guidance, the system provides caregivers with an interpretable sensory load assessment for each video, presenting potentially overwhelming content in a form accessible to non-specialist users.

The main limitation in this approach comes from the trade-off between efficiency and accuracy. A lightweight saliency model runs in real time on mobile devices, but it is less accurate than heavier transformer-based models. The system also uses only visual input, so auditory cues are ignored. This becomes a problem in cases where sound drives attention. These limits come mainly from the requirement for local execution on low-cost hardware and the focus on privacy and accessibility. That constraint is what pushed the design toward a simpler model and pipeline.

There is room for improvement. Adding other modalities like audio or facial recognition could improve performance. The sensory analysis part could also be made more adaptive by personalizing it for the child instead of using fixed parameters. At the moment, the system works as a cold-start version that can be refined once more data is collected.

Overall, this work demonstrates the feasibility of deploying visual attention modelling in a practical assistive application for ASD. While there are clear opportunities for improvement, the system provides a foundation for future research and development of accessible, privacy-preserving assistive technologies in real-world settings.

Bibliography

- [1] Centers for Disease Control and Prevention, *Data and statistics on autism spectrum disorder*, <https://www.cdc.gov/autism/data-research/index.html>, Accessed: 2026-05-25, 2023.
- [2] E. Fridberg, E. Khokhlovich, and A. Vyshedskiy, “Watching videos and television is related to a lower development of complex language comprehension in young children with autism,” *Healthcare*, vol. 9, no. 4, p. 423, 2021. DOI: 10.3390/healthcare9040423 [Online]. Available: <https://doi.org/10.3390/healthcare9040423>
- [3] B. Ekici, *Bana Biraz Otizmden Bahset*. EKİNOKS YAYINEVİ, 2019.
- [4] Q. Wang et al., “Social scene manipulation through gaze-contingent interfaces: Towards automated gaze strategy instruction for young children with asd,” in *Proceedings of the International Meeting for Autism Research (INSAR)*, Baltimore, MD, USA, 2016.
- [5] Q. Wang et al., “Promoting social attention in 3-year-olds with asd through gaze-contingent eye tracking,” *Autism Research*, vol. 13, no. 1, pp. 61–73, 2020. DOI: 10.1002/aur.2199
- [6] E. J. Marco, L. B. Hinkley, S. S. Hill, and S. S. Nagarajan, “Sensory processing in autism: A review of neurophysiologic findings,” *Pediatric Research*, vol. 69, no. 5, 48R–54R, 2011.
- [7] O. Bogdashina, *Sensory Perceptual Issues in Autism and Asperger Syndrome: Different Sensory Experiences-Different Perceptual Worlds*. Jessica Kingsley Publishers, 2016.
- [8] L. Itti, C. Koch, and E. Niebur, “A model of saliency-based visual attention for rapid scene analysis,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 20, no. 11, pp. 1254–1259, 1998.
- [9] K. I. Boser, M. S. Goodwin, and S. C. Wayland, *Technology and Autism Spectrum Disorders*. Brookes Publishing Company, 2014.
- [10] D. W. Sosnowski et al., “Brief report: A novel digital therapeutic that combines applied behavior analysis with gaze-contingent eye tracking to improve emotion recognition in children with autism spectrum disorder,” *Journal of Autism and Developmental Disorders*, vol. 52, no. 5, pp. 2357–2366, 2022. DOI: 10.1007/s10803-021-05116-4

- [11] A. Moskalenko, A. Bryncev, D. Vatolin, R. Timofte, et al., “AIM 2024 challenge on video saliency prediction: Methods and results,” in *Proceedings of the European Conference on Computer Vision (ECCV) Workshops*, 2024.
- [12] B. Zaman, M. Nouwen, J. Vanattenhoven, E. de Ferrer, and J. Van Looy, “Parental mediation of young children’s digital media use,” *Pediatrics*, vol. 137, no. Supplement 2, 2016.
- [13] A. Nawaz, N. Skjæret-Maroni, J. L. Helbostad, B. Vereijken, E. Boulton, and O. Sletvold, “Usability and accessibility of mobile apps for autism: A review,” *Journal of Ambient Intelligence and Humanized Computing*, 2015.
- [14] K. Sowndhararajan, C. Wang, and S. Kim, “Influence of audiovisual stimuli on cognitive load and sensory overload: An eeg and editing-pacing analysis,” *International Journal of Environmental Research and Public Health*, vol. 18, no. 14, p. 7512, 2021. DOI: 10.3390/ijerph18147512
- [15] G. Dillon, “The aesthetics of mindfulness: Analyzing sensory-friendly pacing and emptiness (Ma) in the animation of hayao miyazaki,” *Journal of Animation and Media Studies*, vol. 15, no. 2, pp. 104–122, 2022.
- [16] A. T. Bazzano, C. Wolfe, and Y. Sun, “The impact of sensory-friendly environmental adaptations on media consumption for individuals with autism spectrum disorder,” *Journal of Autism and Developmental Disorders*, vol. 50, no. 4, pp. 1291–1303, 2020. DOI: 10.1007/s10803-019-04312-3
- [17] J. S. Perry and W. S. Geisler, “Gaze-contingent real-time simulation of arbitrary visual fields,” in *Human Vision and Electronic Imaging VII*, SPIE, vol. 4662, 2002, pp. 57–69.
- [18] A. Lang, “The limited capacity model of mediated message processing,” *Journal of Communication*, vol. 50, no. 1, pp. 46–70, 2000.